

Neural Data Science

Lecturer: Dr. Jan Lause, Prof. Dr. Philipp Berens

Tutors: Jonas Beck, Fabio Seel, Julius Würzler

Summer term 2025

Student names: Maria Elisabeth Dirnberger, Akseli Ilmanen, Caroline Griebhaber

LLM Disclaimer: For plots and exploring the GPFA repo.

Coding Lab 6

In this exercise we are going to fit a latent variable model (Poisson GPFA) to both toy data and real data from monkey primary visual cortex. For details, see [Ecker et al. 2014](#) (00104-4.pdf).

Preliminaries

1. Code

The toolbox we are going to use contains an implementation of the EM algorithm to fit the poisson-gpfa.

Assuming you `git clone https://github.com/berenslab/poisson-gpfa` to the parent directory and have the following directory structure:

```
├── data/
│   └── nds_cl_6_data.mat
├── poisson-gpfa/
├── notebooks
│   └── CodingLab6.ipynb
├── matplotlib_style.txt
└── requirements.txt
```

then you can import the related functions via:

```
import sys
sys.path.append('../poisson-gpfa/')
sys.path.append('../poisson-gpfa/funs')

import funs.util as util
import funs.engine as engine
```

Change the paths if you have a different directory structure. For the details of the algorithm, please refer to the thesis [hooram_thesis.pdf](#) from ILIAS.

2. Data

Download the data file `nds_cl_6_data.mat` from ILIAS and save it in a `data/` folder.

```
In [2]: import numpy as np
import scipy.io as sio
import matplotlib.pyplot as plt

# style
import seaborn as sns

# poisson-gpfa
import sys

sys.path.append("../poisson-gpfa/")
sys.path.append("../poisson-gpfa/funs")

import funs.util as util
import funs.engine as engine

%matplotlib inline

%load_ext jupyter_black

%load_ext watermark
%watermark --time --date --timezone --updated --python --iversions --waterma
```

Last updated: 2025-06-01 14:39:44W. Europe Summer Time

Python implementation: CPython
Python version : 3.10.0
IPython version : 8.35.0

sklearn: 1.6.1

scipy : 1.15.2
matplotlib: 3.10.1
numpy : 2.2.4
seaborn : 0.13.2
sys : 3.10.0 | packaged by conda-forge | (default, Nov 20 2021, 02:18:13) [MSC v.1916 64 bit (AMD64)]

Watermark: 2.5.0

```
In [3]: plt.style.use("../matplotlib_style.txt")
```

Task 1. Generate some toy data to test the poisson-GPFA code

We start by verifying our code on toy data. The cell below contains code to generate data for 30 neurons, 100 trials (1000 ms each) and 50ms bin size. The neurons' firing rate λ_k is assumed to be a constant d_k modulated by a one-dimensional latent state x , which is drawn from a Gaussian process:

$$\lambda_k = \exp(c_k x + d_k)$$

Each neuron's weight c_k is drawn randomly from a normal distribution and spike counts are sampled from a Poisson distribution with rate λ_k .

Your task is to fit a Poisson GPFA model with one latent variable to this data (see `engine.PPGPFAfit`).

Hint: You can use `util.dataset?` , `engine.PPGPFAfit?` or `util.initializeParams?` to find out more about the provided package.

Grading: 3 pts

```
In [4]: # -----  
# simulate a training set (0.5 pts)  
  
# Initialize random number generator  
seed = 42  
  
# Specify dataset & fitting parameters  
xdim = 1 # dim of latent state  
ydim = 30 # number of neurons  
n_trials = 100  
trial_dur = 1000 # ms  
bin_size = 50 # ms  
  
# Generate toy dataset using util.dataset  
training_set = util.dataset(  
    trialDur=trial_dur,  
    binSize=bin_size,  
    numTrials=n_trials,  
    xdim=xdim, # dim of latent state  
    ydim=ydim,  
    seed=seed,  
    # dOffset=1.5,  
    # fixTau=True,  
    # fixedTau=np.array([0.2]),  
)  
  
# training_set.avgFR
```

```

+----- Simulated Dataset Options -----+
tent State                                1 | Dimensionality of La
served State (# neurons)                 30 | Dimensionality of Ob
(ms):                                   1000 | Duration of trials
                                         50 | Size of bins (ms):
                                         100 | Number of Trials
+-----+
Sampling trial 100 ...
Average firing rate per neuron in this dataset: 3.480 Hz.

```

Fit the model

```

In [5]: # -----
# fit the model (0.5 pts)
# -----

# Initialize parameters using Poisson-PCA
params_init = util.initializeParams(xdim, ydim, training_set)

# choose sensible parameters and run fit
fitToy = engine.PPGPFAfit(training_set, params_init)

```

Initializing parameters with Poisson-PCA..

```

+----- Fit Options -----+
tent State                                1 | Dimensionality of La
served State (# neurons)                 30 | Dimensionality of Ob
                                         Online | EM mode:
                                         50 | Max EM iterations:
laplace | Inference Method
`diag` | Online Param Update

Method                                   5 | Batch size (trials):
+-----+
Iteration: 50 of 50, nPLL: = -257.9930
This dataset is a simulated dataset.
Processing performance against ground truth parameters...

```

```

In [6]: # some useful functions
def allTrialsState(fit: engine.PPGPFAfit, p) -> np.ndarray:
    """Reshape the latent signal and the spike counts"""
    x = np.zeros([p, 0])
    for i in range(len(fit.infRes["post_mean"])):
        x = np.concatenate((x, fit.infRes["post_mean"][i]), axis=1)
    return x

def allTrialsX(training_set: util.dataset) -> np.ndarray:
    """Reshape the ground truth
    latent signal and the spike counts"""

```

```
x_gt = np.array([])
for i in range(len(training_set.data)):
    x_gt = np.concatenate((x_gt, training_set.data[i]["X"][0]), axis=0)
return x_gt
```

Plot the ground truth vs. inferred model

Verify your fit by plotting both ground truth and inferred parameters for:

1. weights C
2. biases d
3. latent state x

Note that the sign of fitted latent state and its weights are ambiguous (you can flip both without changing the model). Make sure you correct the sign for the plot if it does not match the ground truth.

```
In [7]: # All trials latent state vector
x_est = allTrialsState(fitToy, 1)
x_true = allTrialsX(training_set)
x_est.shape, x_true.shape
```

```
Out[7]: ((1, 100), (2000,))
```

```
In [8]: # -----
# Plot ground truth vs. inferred model
# Plot the weights `C`, biases `d` and latent states (2 pts)
# -----
x_est = x_est.flatten()

# all shape: (n_neurons,)
C_true = training_set.params["C"].flatten()
d_true = training_set.params["d"]

C_est = fitToy.optimParams["C"].flatten()
d_est = fitToy.optimParams["d"]

sign_flip = -1

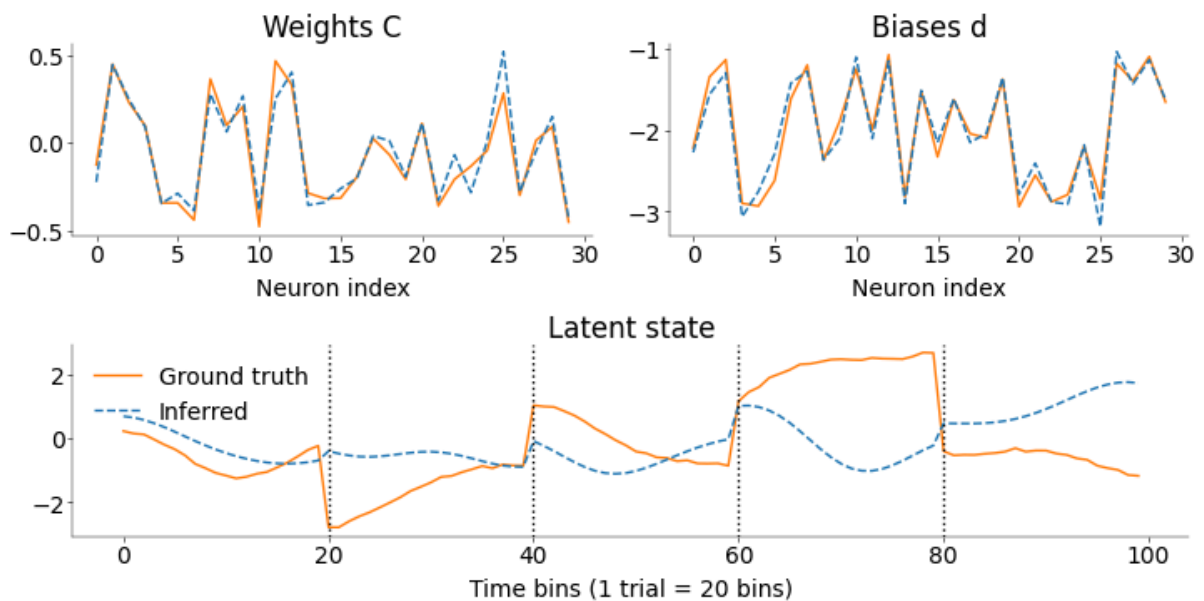
# add plot
fig, ax = plt.subplot_mosaic([["C", "d"], ["latent", "latent"]])
# For d & C consider also plotting the optimal weights as a dotted line for

ax["C"].plot(C_true, label="Ground truth", color="C1")
ax["C"].plot(sign_flip * C_est, label="Inferred", color="C0", linestyle="--")
ax["C"].set_title("Weights C")
ax["C"].set_xlabel("Neuron index")

ax["d"].plot(d_true, label="Ground truth", color="C1")
ax["d"].plot(d_est, label="Inferred", color="C0", linestyle="--")
ax["d"].set_title("Biases d")
ax["d"].set_xlabel("Neuron index")
```

```
# For the latent states consider separating each trial by a vertical line
# plot only for a subset of trials
ax["latent"].plot(x_true[: x_est.size], label="Ground truth", color="C1")
ax["latent"].plot(sign_flip * x_est, label="Inferred", color="C0", linestyle="dashed")
ax["latent"].set_title("Latent state")
ax["latent"].set_xlabel("Time bins (1 trial = 20 bins)")
ax["latent"].legend()

for i in range(1, 5):
    ax["latent"].axvline(i * 20, color="k", linestyle=":")
```



In []:

In []:

In []:

Task 2: Fit GPFA model to real data.

We now fit the model to real data and cross-validate over the dimensionality of the latent variable.

Grading: 4 pts

Load data

The cell below implements loading the data and encapsulates it into a class that matches the interface of the Poisson GPFA engine. You don't need to do anything here.

```
In [9]: class EckerDataset:
        """Loosy class"""

        def __init__(
            self,
            path: str,
            subject_id: int = 0,
            ydim: int = 55,
            trialDur: int = 2000,
            binSize: int = 100,
            numTrials: int = 100,
            ydimData: bool = False,
            numTrData: bool = True,
        ):
            # T = binSize #int(trialDur/binSize)
            T = int(trialDur / binSize)
            matdat = sio.loadmat(path)
            self.matdat = matdat
            data = []
            trial_durs = []
            for trial_id in range(numTrials):
                trial_time = matdat["spikeTimes"][:, trial_id][0]
                trial_big_time = np.min(trial_time)
                trial_end_time = np.max(trial_time)
                trial_durs.append(trial_end_time - trial_big_time)
            for trial_id in range(numTrials):
                Y = []
                spike_time = []
                data.append(
                    {
                        "Y": matdat["spikeCounts"][:, :, trial_id],
                        "spike_time": matdat["spikeTimes"][:, trial_id],
                    }
                )
            self.T = T
            self.trial_durs = trial_durs
            self.data = data
            self.trialDur = trialDur
            self.binSize = binSize
            self.numTrials = numTrials
            self.ydim = ydim
            util.dataset.getMeanAndVariance(self)
            util.dataset.getAvgFiringRate(self)
            util.dataset.getAllRaster(self)
```

```
In [10]: path = "../data/nds_cl_6_data.mat"
        data = EckerDataset(path)
```

Fit Poisson GPFA models and perform model comparison

Split the data into 80 trials used for training and 20 trials held out for performing model comparison. On the training set, fit models using one to five latent variables. Compute the performance of each model on the held-out test set.

Hint: You can use the `crossValidation` function in the Poisson GPFA package.

Optional: The `crossValidation` function computes the sum of the squared errors (SSE) on the test set, which is not ideal. The predictive log-likelihood under the Poisson model would be a better measure, which you are welcome to compute instead.

Derivation for log-likelihood

Probability for an event (here spike x_t) under a Poisson distribution is given by:

$$p_\lambda(x_t) = \frac{\lambda^{x_t} e^{-\lambda}}{x_t!}$$

If we assume that the spike events are independent from each other, we can define the likelihood function as a product of the independent observations $x_1, \dots, x_N$:

$$L(\lambda; x_1, \dots, x_N) = \prod_{t=1}^N \frac{\lambda^{x_t} e^{-\lambda}}{x_t!}$$

Using the log product rule, we can simplify this:

$$\log L(\lambda; x_1, \dots, x_N) = \log\left(\prod_{t=1}^N \frac{\lambda^{x_t} e^{-\lambda}}{x_t!}\right)$$

$$\log L(\lambda; x_1, \dots, x_N) = \sum_{t=1}^N [\log(\lambda^{x_t}) + \log(e^{-\lambda}) - \log(x_t!)]$$

$$\log L(\lambda; x_1, \dots, x_N) = \sum_{t=1}^N [x_t \log \lambda - \lambda - \log(x_t!)]$$

```
In [30]: ## -----  
## Perform cross validation (1 pt)  
## -----  
  
fitEcker = util.crossValidation(  
    experiment=data,  
    numTrainingTrials=80,  
    numTestTrials=20,  
    maxXdim=5,  
    learningMethod="batch",  
) # batch, diag, hess, grad  
  
# do the actual cross validation
```



```
xval = fitEcker.optimXdim  
print()  
print()  
print(f"Optimal latent state {xval}")
```

Assessing optimal latent dimensionality will take a long time.

Initializing parameters with Poisson-PCA..

+----- Fit Options -----+

1 | Dimensionality of Latent State

55 | Dimensionality of Observed State (# neurons)

Batch | EM mode:

3 | Max EM iterations:

laplace | Inference Method

+-----+

Iteration: 3 of 3, nPLL: = -406.2404 Performing leave-one-out cross validation...

Initializing parameters with Poisson-PCA..

+----- Fit Options -----+

2 | Dimensionality of Latent State

55 | Dimensionality of Observed State (# neurons)

Batch | EM mode:

3 | Max EM iterations:

laplace | Inference Method

+-----+

Iteration: 3 of 3, nPLL: = -370.8867 Performing leave-one-out cross validation...

Initializing parameters with Poisson-PCA..

+----- Fit Options -----+

3 | Dimensionality of Latent State

55 | Dimensionality of Observed State (# neurons)

Batch | EM mode:

3 | Max EM iterations:

laplace | Inference Method

+-----+

Iteration: 3 of 3, nPLL: = -376.9620 Performing leave-one-out cross validation...

Initializing parameters with Poisson-PCA..

+----- Fit Options -----+

4 | Dimensionality of Latent State

55 | Dimensionality of Observed State (# neurons)

Batch | EM mode:

3 | Max EM iterations:

laplace | Inference Method

+-----+

Iteration: 3 of 3, nPLL: = -363.3572 Performing leave-one-out cross validation...

Initializing parameters with Poisson-PCA..

+----- Fit Options -----+

5 | Dimensionality of Latent State

55 | Dimensionality of Observed State (# neurons)

Batch | EM mode:

3 | Max EM iterations:
laplace | Inference Method

+-----+
Iteration: 3 of 3, nPLL: = -359.8685Performing leave-one-out cross validation...

Optimal latent state 4

Plot the test error

Make a plot of the test error for the five different models. As a baseline, please also include the test error of a model without a latent variable. This is essentially the SSE of a constant rate model (or Poisson likelihood if you did the optional part above). Note: We assume a constant firing rate across trials, but not necessarily across time.

```
In [12]: # -----  
# Compute and plot the test errors for the different latent variable models  
# -----  
  
def concat(data):  
    n_trials = data.numTrials  
    n_neurons = data.ydim  
    n_bins = data.data[0]["Y"].shape[1]  
  
    data_concat = np.zeros((n_trials, n_neurons, n_bins))  
  
    for t, trial in enumerate(data.data):  
        data_concat[t, :, :] = trial["Y"]  
  
    return data_concat  
  
train, test = util.splitTrainingTestDataset(  
    data, numTrainingTrials=80, numTestTrials=20  
)  
  
train_concat = concat(train)  
test_concat = concat(test)  
  
spikeCountsMeans = np.mean(train_concat, axis=0) # average  
  
baseline_error = ((spikeCountsMeans - test_concat) ** 2).sum()
```

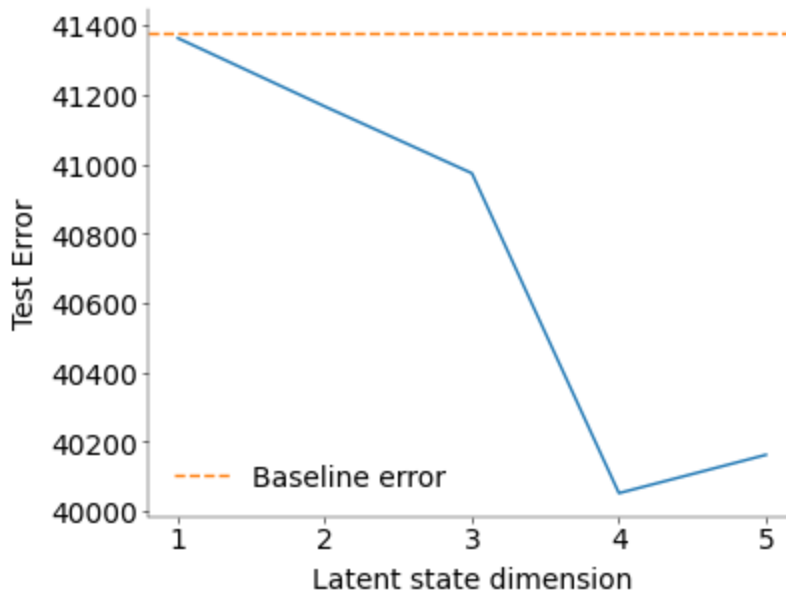
```
In [16]: # -----  
# Compute and plot the test errors for the different latent variable models  
# and answer the questions below (1+1 pts)  
# -----  
  
# Your plot here
```

```
fig, ax = plt.subplots(figsize=(4, 3))

# plot model error
errors = fitEcker.errs

# plot baseline
ax.axhline(baseline_error, linestyle="--", label="Baseline error", color="C1")
ax.plot([1, 2, 3, 4, 5], errors)
plt.ylabel("Test Error")
plt.xlabel("Latent state dimension")
plt.legend()
```

Out[16]: <matplotlib.legend.Legend at 0x1a12fcb3e80>



Questions

Is the baseline error (0 latents) about what you expect in relation to the other models? Why or why not?

I would have expected that the test error would be quite a bit lower for a single latent dimension compared to baseline (no latents). This is even more surprising given that in the Raster + Latent plots later, I can see a clear relationship between the latent and spike counts.

Can you identify a "best model". If so, which is it and what does this say about the structure of the latent state?

Based on the SSE, it seems 4 latent dimensions seems best. However, I noticed that in the print output of `util.crossValidation` the `nPLL` output kept decreasing even for 5 dimensions.

Task 3. Visualization: population rasters and latent state.

Use the model with a single latent state.

Create a raster plot where you show for each trial the spikes of all neurons as well as the trajectory of the latent state `x` (take care of the correct time axis). Sort the neurons by their weights `c_k`. Plot only the first 20 trials.

Grading: 2 pts

```
In [14]: from numpy import matplotlib
import funs.util as util

# Your plot here
fig, axs = plt.subplots(10, 2, figsize=(14, 14))

ts = np.linspace(0, 2000, 100)
xa = 0.15
xs = 0.7 * xa * np.sin(ts / 1000 * 3.4 * 2 * np.pi) + xa

def myraster(event_times_list, color="k", ax=None):
    for ith, trial in enumerate(event_times_list):
        ax.vlines(trial, ith + 0.5, ith + 1.5, color=color)

C = fitEcker.fits[0].optimParams["C"].flatten()
sorted_idx = np.argsort(C)

with sns.axes_style("ticks"):
    for ntrial, ax in enumerate(axs.flat):

        x = range(50, 2000, 100) # assume binsize of 100ms

        # -----
        # plot the raster for each neuron and latent state (2 pts)
        x_est_trial = fitEcker.fits[0].infRes["post_mean"][ntrial].flatten()
        ax.plot(
            x, sign_flip * x_est_trial * 10 - 15, color="red", label="Latent"
        ) # sign_flip so neural bursts aligned with increase in latent state
        ax.set_ylabel("Neuron IDs")

        spike_times = event_times_list = data.data[ntrial]["spike_time"]
        sorted_spike_times = spike_times[sorted_idx]

        myraster(sorted_spike_times, color="k", ax=ax)

        # -----

        # hint: can be plotted on top of the corresponding raster
```

```

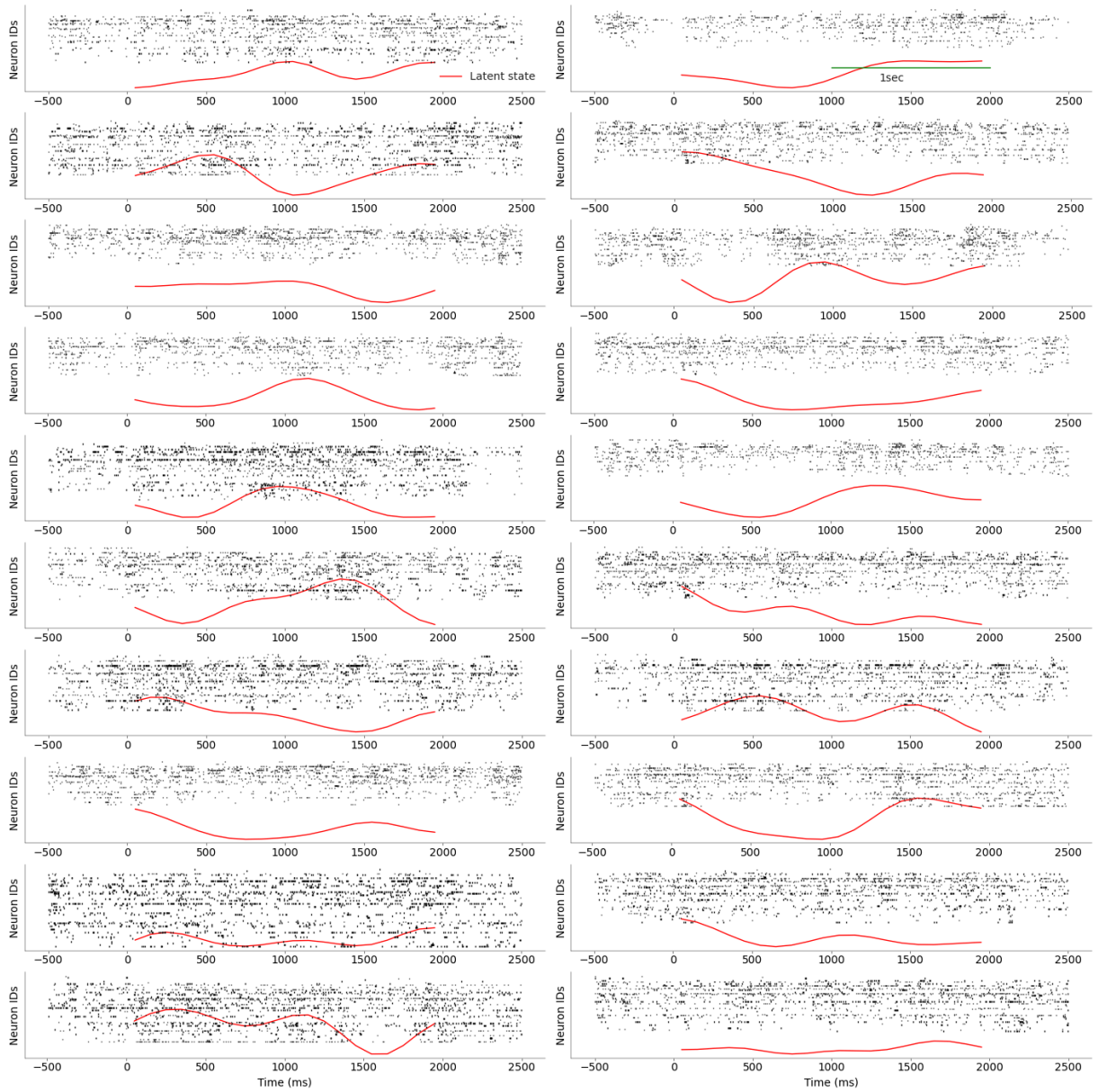
# sort neurons by weight

if ntrial == 0:
    ax.legend(loc="lower right")
if ntrial == 1:
    ax.plot([1000, 2000], [-30, -30], color="green")
    ax.text(1300, -50, "1sec")
if ntrial < 2:
    # plot a wavy line at the top for the last two subplots
    if ntrial >= len(axes.flat) - 2:
        ax.plot(ts, (xs * 40) + data.ydim, color="black")

# plot xlabel on the last two
if ntrial >= len(axes.flat) - 2:
    ax.set_xlabel("Time (ms)")
else:
    ax.set_xlabel("")

ax.set_yticks([])
# ax.set_xticks([])

```



Interpretation

The latent state (after sign flip) seems to have higher values in time bins where the spike count is higher, and one can observe some neuronal bursting.

Task 4. Visualization of covariance matrix.

Plot (a) the covariance matrix of the observed data as well as its approximation using (b) one and (c) five latent variable(s). Use the analytical solution for the covariance matrix of the approximation*. Note that the solution is essentially the mean and covariance of the [log-normal distribution](#).

$$\mu = \exp\left(\frac{1}{2} \text{diag}(CC^T) + d\right)$$

$$\text{Cov} = \mu \otimes \mu^T \odot \exp(CC^T) + \mu \cdot \mathbb{I} - \mu \otimes \mu^T$$

*Krumin, M., and Shoham, S. (2009). Generation of Spike Trains with Controlled Auto- and Cross-Correlation Functions. *Neural Computation* 21, 1642–1664.

Grading: 3 pts

```
In [ ]: # -----
# Complete the analytical solution for the covariance matrix of
# the approximation using the provide equations (2 pts)
# -----

def cov(fit: engine.PPGPFAfit) -> np.ndarray:

    C = fit.optimParams["C"]
    d = fit.optimParams["d"]

    # np.diag(C @ C.T) same as C**2 (element-wise square of C)
    mu = np.exp(0.5 * np.diag(C @ C.T) + d)
    mu = np.reshape(mu, (mu.shape[0], 1)) # (55, 1)

    c = (
        np.multiply(np.kron(mu, mu.T), np.exp(C @ C.T))
        + mu * np.eye(mu.shape[0])
        - np.kron(mu, mu.T)
    )

    return c, mu

# -----
# Plot the covariance matrix of
# (1) the observed data
# (2) its approximation using 1 latent variable
# (3) its approximation using 5 latent variable
# and explain how they compare (1+1 pts).
# -----

obs_corr = np.cov(data.all_raster)
opt_r1, mu1 = cov(fitEcker.fits[0])
opt_r5, mu5 = cov(fitEcker.fits[4])

# sort the covariance matrices by the weights (sorted_idx from above)
obs_corr = obs_corr[sorted_idx, :][:, sorted_idx]
opt_r1 = opt_r1[sorted_idx, :][:, sorted_idx]
opt_r5 = opt_r5[sorted_idx, :][:, sorted_idx]

# HINT: Think about which type of colormap and ranges are appropriate here.

fig, axs = plt.subplots(1, 3, figsize=(10, 3.5))
# add plot to visualize the differences in the covariance matrices

sns.heatmap(
    obs_corr,
```



```

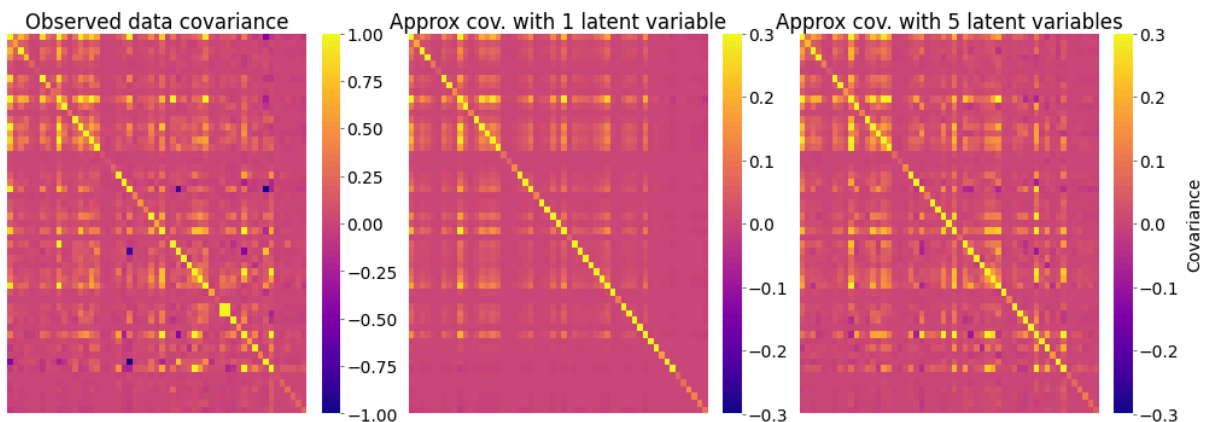
ax=axes[0],
cmap="plasma",
vmin=-1,
vmax=1,
xticklabels=False,
yticklabels=False,
)
axes[0].set_title("Observed data covariance")

sns.heatmap(
    opt_r1,
    ax=axes[1],
    cmap="plasma",
    vmin=-0.3,
    vmax=0.3,
    xticklabels=False,
    yticklabels=False,
)
axes[1].set_title("Approx cov. with 1 latent variable")

sns.heatmap(
    opt_r5,
    ax=axes[2],
    cmap="plasma",
    cbar_kws={"label": "Covariance"},
    vmin=-0.3,
    vmax=0.3,
    xticklabels=False,
    yticklabels=False,
)
axes[2].set_title("Approx cov. with 5 latent variables")

```

Out[]: Text(0.5, 1.0, 'Approx cov. with 5 latent variables')



Questions

What do you see / expect to see?

First of all, we notice that the observed covariance matrix looks much closer to the approximated one when using latent variables (instead of 1). This is line with our earlier findings. It's also noteworthy that besides the high values on the

diagonal (to be expected), there are some quadrants of high covariance. This suggests that neurons who are in the same quadrant belong to a similar state in the Gaussian process. Given that the Gaussian process modelled here is over the continuous domain of time, we can interpret this as the following: The latent state for neurons in the same quadrant is temporally smooth. To highlight this quadrant effect, we also sorted the neuron IDs by their weights C (as was this in the Raster plots).

It's also noteworthy that the observed data covariance and approximated covariance (dim=5) looks more similar to each other when using a smaller range colourmap range for the estimated one. @Tutor, I am not sure why this is, but would love to hear your interpretation of this.